GOLDEN GATE UNIVERSITY

San Francisco, California

CYBER EVENT CLASSIFICATION USING MACHINE LEARNING TECHNIQUES

A Capstone Project Report Submitted in Partial Fulfillment of the Requirements for Data110

Submitted by

Afifa Sabha
Satvik Kumar

Base Paper: Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.

August 2026

CERTIFICATE

This is to certify that the capstone project report entitled “Cyber Event Classification Using Machine Learning Techniques” is a bona fide record of work carried out by Afifa Sabha and Satvik Kumar, submitted in partial fulfillment of the requirements for the course Data110 at Golden Gate University, San Francisco.

The work presented in this report is original and has been carried out under the guidance of the course instructor. It has not been submitted elsewhere for the award of any other degree or diploma.

Course Instructor

Data110, Golden Gate University

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our course instructor for their invaluable guidance, encouragement, and support throughout the duration of this capstone project. Their feedback at every stage helped shape the direction of this work.

We are also thankful to Golden Gate University for providing the academic environment, resources, and infrastructure necessary to carry out this project. Finally, we thank our peers and family for their continuous support and encouragement.

Afifa Sabha
Satvik Kumar

ABSTRACT

Cybersecurity incidents continue to grow in scale and sophistication, making automated classification of attack types an important tool for security operations teams. This capstone project develops and evaluates a complete machine learning pipeline for classifying the type of a cyber-attack (attack_type) from a synthesized cybersecurity events dataset comprising 100,000 records and 15 attributes. Building on the ensemble learning methodology introduced by Breiman (2001), we implement and compare four classification approaches: Logistic Regression, Decision Tree, Random Forest, and an Artificial Neural Network (ANN).

The pipeline includes systematic data cleaning, exploratory data analysis, feature engineering (temporal feature extraction, one-hot encoding, and feature scaling), an 80/20 stratified train-test split, hyperparameter tuning via GridSearchCV, and 5-fold stratified cross-validation. Models are evaluated using Accuracy, Precision, Recall, F1 Score, and ROC-AUC.

Experimental results show that all four models achieve accuracy close to the theoretical random-guessing baseline of approximately 12.5% for this 8-class problem, with Random Forest performing marginally best (12.90% test accuracy). Exploratory analysis reveals that the dataset's features are distributed almost uniformly with respect to the target variable, indicating limited class-discriminative signal in this particular synthesized dataset. Rather than treating this as a failure, the project documents this finding transparently as a key methodological insight, and discusses its implications for future work using real-world security telemetry. The complete, reproducible pipeline, trained models, and evaluation artifacts are made available in an accompanying GitHub repository.

Keywords

Cybersecurity, Machine Learning, Random Forest, Decision Tree, Logistic Regression, Artificial Neural Network, Multiclass Classification, Feature Engineering, Cyber Event Classification, Intrusion Detection.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1: INTRODUCTION

Cyberattacks represent one of the most persistent and evolving threats to modern digital infrastructure. Organizations across every industry face a continuous stream of security incidents ranging from phishing and malware to distributed denial-of-service (DDoS) attacks, ransomware, and zero-day exploits. Given the sheer volume of security events generated in modern networks, manual triage and classification of incidents is no longer feasible at scale. This has motivated a large and growing body of research applying machine learning (ML) techniques to automate the detection and classification of cyber-attacks.

Machine learning offers the ability to learn complex, non-linear relationships between observable attack characteristics (such as duration, data compromised, targeted system, and response time) and the underlying category of attack. Ensemble methods, in particular Random Forest as introduced by Breiman (2001), have become a standard baseline in cybersecurity classification research due to their strong empirical performance, robustness to noisy or irrelevant features, and interpretability through feature importance scores.

This capstone project applies a complete, end-to-end machine learning workflow to a synthesized cybersecurity events dataset, with the goal of classifying the type of attack represented by each record. The project follows established data science methodology: data cleaning, exploratory data analysis (EDA), feature engineering, model development, hyperparameter tuning, cross-validation, and rigorous multi-metric evaluation.

1.1 Problem Statement

Given a dataset of recorded cybersecurity incidents described by attack characteristics, target system information, organizational context, and response metrics, the problem addressed in this project is: can a supervised machine learning model accurately predict the type of cyber-attack (attack_type) associated with a given incident record? This is formulated as a multiclass classification problem with eight target classes: Phishing, DDoS, Zero-Day, Ransomware, Malware, SQL Injection, Brute Force, and XSS.

1.2 Objectives

- To perform comprehensive data cleaning and exploratory data analysis on the cybersecurity events dataset.
- To engineer relevant features, including temporal and derived features, suitable for multiclass classification.
- To implement and train four classification algorithms: Logistic Regression, Decision Tree, Random Forest, and Artificial Neural Network.
- To tune model hyperparameters using GridSearchCV and validate model stability using k-fold cross-validation.
- To evaluate and compare all models using Accuracy, Precision, Recall, F1 Score, and ROC-AUC.
- To identify the best-performing model and provide a transparent, data-driven justification for the selection.
- To honestly document any limitations discovered in the dataset or modeling process.

CHAPTER 2: LITERATURE REVIEW

This chapter reviews prior research relevant to machine learning-based cybersecurity event and intrusion classification, organized around the base paper, ensemble learning theory, benchmark datasets, and application-specific studies.

2.1 Foundational Work: Random Forests

Breiman (2001) introduced Random Forests as an ensemble learning method that constructs a multitude of decision trees during training and outputs the class that is the mode of the individual trees' predictions. The method combines bagging with random feature selection at each split, which reduces variance and correlation among trees, resulting in improved generalization compared to a single decision tree. This foundational paper underpins the majority of tree-ensemble approaches used in the cybersecurity classification literature reviewed below.

2.2 Machine Learning Surveys in Cybersecurity

Alzahrani, Alarifi, and Altamimi (2020) surveyed the application of machine learning techniques across the cybersecurity domain, highlighting that ensemble methods and neural networks are the most widely applied approaches for intrusion and malware detection tasks, while noting persistent challenges around dataset quality, class imbalance, and generalization to real-world traffic.

Gibert, Mateu, and Planes (2020) reviewed the evolution of machine learning for malware detection and classification, tracing the shift from traditional feature-engineered classifiers toward deep learning approaches, and identifying research gaps around adversarial robustness and the scarcity of large, labeled, real-world malware datasets.

2.3 Random Forest and Ensemble Approaches for Attack Detection

Alharbi and Islam (2019) applied a Random Forest ensemble combined with feature selection for network intrusion detection, reporting that careful feature selection substantially improved classification performance and reduced training time relative to using the full feature set.

Wang, Li, and Huang (2020) proposed a Random Forest-based approach for botnet detection, demonstrating that ensemble tree methods could effectively separate botnet traffic from benign traffic using flow-level statistical features.

Raza, Sheikh, Hwang, and Ab-Rahman (2024) examined feature-selection-based DDoS attack detection using multiple AI algorithms, finding that reducing the feature space to the most informative attributes improved both detection accuracy and computational efficiency for real-time deployment scenarios.

Fathima, Devi, and Faizaanuddin (2023) investigated supervised machine learning methods for improving DDoS attack detection, comparing several classifiers and confirming that ensemble-based models consistently outperformed single-classifier baselines on their benchmark traffic dataset.

2.4 Comparative Studies of Classical ML Classifiers

A comparative study evaluating Random Forest, Logistic Regression, and Neural Networks for cybersecurity attack classification found that while all three approaches achieved reasonable performance, Random Forest offered the most favorable balance between predictive accuracy and interpretability across the datasets tested.

Similarly, a study evaluating Support Vector Machine, Logistic Regression, and Random Forest classifiers for cyberattack detection found that combining Logistic Regression and Random Forest through a voting classifier, together with information-gain-based feature selection, improved detection performance over any single algorithm.

Manita and Suleiman (2024) trained several machine learning algorithms specifically for detecting distributed denial-of-service attacks, reporting that model performance was highly sensitive to the quality and representativeness of the underlying training data — a finding directly relevant to the present project's own data-quality observations.

2.5 Deep Learning and Hybrid Approaches

Halbouni et al. (2022) proposed a hybrid CNN-LSTM architecture for network intrusion detection, showing that combining convolutional feature extraction with sequential modeling could capture both spatial and temporal patterns in network traffic, outperforming standalone CNN or LSTM models on benchmark intrusion datasets.

A broader comparative study of deep learning versus classical machine learning for intrusion detection found that while deep architectures (MLP, CNN, LSTM) can outperform classical models on large, well-structured benchmark datasets, they also require substantially more data and computational resources, and are more sensitive to class imbalance without techniques such as SMOTE.

He, Kim, and Asghar (2023) surveyed adversarial machine learning in the context of network intrusion detection systems, cautioning that high benchmark accuracy does not guarantee robustness, and that models trained on clean, well-separated benchmark data may not generalize to adversarial or noisy real-world conditions — a caution echoed by the near-random performance observed in the present project on a synthetic, feature-uninformative dataset.

2.6 Application-Specific Studies: Phishing Detection

A study evaluating machine learning and neural network models for phishing detection compared traditional classifiers against RNN, LSTM, and GRU architectures on email data, finding that while deep sequential models captured complex patterns more effectively, simpler models such as Logistic Regression remained competitive and far cheaper to train and deploy (Rangelov, Miltchev, & Genchev, 2026).

Kapan and Sora Gunal (2023) conducted a comprehensive evaluation of classifiers and features for phishing attack detection, concluding that feature selection strategy had a larger impact on detection performance than the specific choice of classification algorithm — a conclusion that parallels this project's own finding that feature quality, not model choice, is the binding constraint on performance.

2.7 Benchmark Datasets

Sharafaldin, Lashkari, and Ghorbani (2018) introduced the CICIDS2017 dataset, which has since become one of the most widely used benchmark datasets for network intrusion detection research, providing labeled flows across multiple attack categories captured in a realistic network environment.

Engelen, Rimmer, and Joosen (2021) conducted a detailed audit of the CICIDS2017 dataset, identifying labeling inconsistencies and feature leakage issues that had previously inflated reported model accuracies in the literature — reinforcing the importance of rigorous dataset auditing, a practice this project also applies to its own (synthesized) dataset in Chapter 4.

2.8 Hybrid and Dependable Models

Talukder, Hasan, Islam, et al. (2023) proposed a dependable hybrid machine learning model for network intrusion detection that combined oversampling with ensemble feature extraction, reporting improved detection of minority attack classes compared to standard single-model baselines.

2.9 Literature Comparison Table

The table below summarizes methodology, dataset, results, and limitations for each reviewed study, using APA 7th edition author-year citation style throughout.

Authors (Year)	Methodology	Dataset Used	Key Results	Limitations
Breiman (2001)	Random Forest ensemble theory	N/A (theoretical)	Ensembles of trees reduce variance vs. single trees	No direct empirical cybersecurity application
Alzahrani et al. (2020)	Survey of ML in cybersecurity	Multiple (survey)	Ensembles & NNs dominate the field	Generalization and data-quality challenges noted, not solved
Gibert et al. (2020)	Survey of ML for malware detection	Multiple (survey)	Field is shifting toward deep learning	Adversarial robustness remains an open gap
Alharbi & Islam (2019)	RF + feature selection	Network intrusion dataset	Feature selection improved RF accuracy and speed	Evaluated on a single dataset; generalization untested
Wang et al. (2020)	RF for botnet detection	Flow-level traffic dataset	RF effectively separated botnet from benign traffic	Dataset scope and diversity limited
Raza et al. (2024)	Feature-selection DDoS detection	DDoS traffic dataset	Reduced feature sets improved accuracy & efficiency	Feature selection method is dataset-specific
Fathima et al. (2023)	Supervised ML for DDoS detection	DDoS traffic dataset	Ensembles outperformed single classifiers	Binary classification only; single dataset
Manita & Suleiman (2024)	ML algorithms for DDoS detection	DDoS traffic dataset	Performance highly sensitive to data quality	Limited discussion of dataset provenance

Authors (Year)	Methodology	Dataset Used	Key Results	Limitations
Halbouni et al. (2022)	Hybrid CNN-LSTM	Benchmark intrusion datasets	Outperformed standalone CNN/LSTM models	Higher computational cost than classical models
He et al. (2023)	Survey of adversarial ML for IDS	Multiple (survey)	High benchmark accuracy does not guarantee robustness	Survey only; no new empirical model proposed
Sharafaldin et al. (2018)	Dataset generation & characterization	CICIDS2017 (introduced)	Introduced a widely-used labeled IDS benchmark	Later found to contain labeling/leakage issues
Engelen et al. (2021)	Dataset audit	CICIDS2017	Identified labeling inconsistencies inflating accuracy	Audit only; does not propose a corrected model
Talukder et al. (2023)	Hybrid oversampling + ensemble	NSL-KDD-type IDS dataset	Improved detection of minority attack classes	Added pipeline complexity from hybrid design
Rangelov et al. (2026)	ML vs. NN comparison	Phishing email dataset	LSTM/GRU capture patterns; simple models still competitive	Deep models costlier; generalization to real-world unclear
Kapan & Sora Gunal (2023)	Classifier & feature evaluation	Phishing URL/website datasets	Feature selection mattered more than algorithm choice	Feature-engineering dependent; not end-to-end automated

2.10 Research Gap

While the reviewed literature demonstrates strong performance of ensemble and hybrid ML models on established benchmark datasets such as CICIDS2017, NSL-KDD, and UNSW-NB15, most studies rely on data with genuine class-conditional structure derived from real or realistically simulated network traffic. Comparatively little published work explicitly documents and analyzes cases where a cybersecurity dataset lacks meaningful predictive signal — a scenario that is nonetheless common in practice when working with synthetic, randomly-generated, or poorly instrumented data sources.

This project addresses that gap by applying the same rigorous ML pipeline used in the reviewed literature to a synthesized dataset, and by explicitly diagnosing and reporting the resulting near-random model performance as a legitimate and instructive finding, following the dataset-auditing precedent set by Engelen et al. (2021). This transparent approach to negative or null results is itself a contribution, illustrating that a technically correct pipeline can and should still report honest outcomes rather than only pursuing high accuracy.

2.11 Novelty of the Proposed Solution

While the studies reviewed above demonstrate strong empirical results on established benchmark datasets, this project's novelty lies not in proposing a new algorithm, but in the rigor and transparency of its evaluation methodology when applied to a dataset whose predictive quality had not been previously audited. Specifically, this project contributes:

- A complete four-model comparison (Logistic Regression, Decision Tree, Random Forest, ANN) applied consistently to the same feature set, preprocessing pipeline, and evaluation protocol — enabling a fair, like-for-like comparison rather than cherry-picking a single best-performing algorithm, as several reviewed studies do.
- An explicit dataset-quality audit (Chapter 4, EDA) performed before model development, in the spirit of Engelen et al. (2021), rather than only after observing poor results.
- A multi-metric, multi-model convergence test: because all four independently-trained models converge to the same near-baseline performance (Chapter 5), the project demonstrates a rigorous way to distinguish a data-quality problem from a model-selection or tuning problem — a diagnostic pattern not explicitly documented in the reviewed cybersecurity ML literature.
- A fully reproducible, version-controlled pipeline (notebook, modular source scripts, saved models, and a GitHub repository) built specifically for this dataset, rather than adapted from an existing Kaggle notebook or published solution.

2.12 How This Methodology Differs From Prior Studies

Most reviewed studies (e.g., Alharbi & Islam, 2019; Halbouni et al., 2022; Talukder et al., 2023) report the performance of one proposed model against a small number of baselines on a single benchmark dataset, typically emphasizing the highest accuracy achieved. This project instead treats model comparison and dataset validation as equally important outputs: Chapter 4's EDA-driven uniformity analysis, Chapter 5's ROC-AUC and error analysis, and Chapter 5's cross-model convergence check are used jointly as evidence for a data-centric conclusion, rather than a purely model-centric one. This reflects a growing but still

under-represented perspective in the cybersecurity ML literature — exemplified by Engelen et al. (2021) — that a dataset's fitness for purpose must be established before its use in benchmarking is meaningful.

CHAPTER 3: PROPOSED METHODOLOGY

This chapter describes the end-to-end methodology followed in this project, covering the dataset, preprocessing pipeline, feature engineering strategy, and modeling approach. The methodology follows the standard supervised machine learning workflow, adapted for a multiclass cybersecurity classification task.

3.1 Dataset Description

The dataset used in this project, cybersecurity_synthesized_data.csv, contains 100,000 records describing simulated cybersecurity incidents, with 15 attributes per record.

Column	Description	Type
attack_type	Category of cyber-attack (target variable, 8 classes)	Categorical
target_system	System type targeted by the attack	Categorical
outcome	Whether the attack succeeded or failed	Categorical (binary)
timestamp	Date and time the incident was recorded	Datetime
attacker_ip / target_ip	Source and destination IP addresses	Identifier
data_compromised_GB	Volume of data compromised, in gigabytes	Numeric
attack_duration_min	Duration of the attack, in minutes	Numeric
security_tools_used	Security tooling active at the time	Categorical
user_role	Role of the affected user account	Categorical
location	Geographic location of the incident	Categorical
attack_severity	Recorded severity rating	Numeric
industry	Industry sector of the targeted organization	Categorical
response_time_min	Time taken to respond to the incident	Numeric
mitigation_method	Method used to mitigate the attack	Categorical

Table 1: Dataset Column Description

3.2 Data Preprocessing

Data preprocessing involved five steps: (1) verifying and handling missing values — none were found; (2) removing duplicate records — none were found; (3) standardizing categorical text formatting by trimming whitespace; (4) converting the timestamp field to a proper datetime type; and (5) applying IQR-based outlier detection on all numeric features. Outliers were retained rather than removed, since extreme values may represent legitimate edge-case incidents in a security context.

3.3 Feature Engineering

Feature engineering steps applied in this project are summarized below.

Step	Technique	Rationale
Identifier removal	Drop attacker_ip, target_ip	Near-unique per record; no generalizable signal
Temporal extraction	hour, day_of_week, month from timestamp	Captures potential time-based attack patterns
Derived feature	is_business_hours flag	Business-hours vs. off-hours attacker

Step	Technique	Rationale
		behavior may differ
Target encoding	Label Encoding on attack_type	Required numeric format for classifiers
Categorical encoding	One-Hot Encoding on 7 nominal columns	Avoids imposing false ordinal relationships
Feature scaling	StandardScaler on numeric columns	Required for Logistic Regression and ANN convergence

Table 3: Feature Engineering Summary

3.4 Train-Test Split

The dataset was split into training (80%) and test (20%) sets using stratified sampling on the target variable, preserving the class balance of attack_type in both partitions. The training set contained 80,000 records and the test set contained 20,000 records.

3.5 Model Development and Justification

Four classification models were implemented, each selected for a distinct methodological reason:

- Logistic Regression — chosen as a linear, probabilistic baseline. It is fast to train, highly interpretable via coefficients, and establishes a lower bound against which more complex models can be judged; if a complex model cannot outperform this baseline, that is itself informative.
- Decision Tree — chosen as a single, fully interpretable non-linear model. It provides a human-readable decision structure and serves as the direct building block for the Random Forest ensemble, making it useful for isolating the marginal benefit of ensembling.
- Random Forest — chosen as the primary candidate model, directly motivated by the base paper (Breiman, 2001). It aggregates many decorrelated decision trees to reduce variance, is robust to noisy or non-informative features, requires minimal preprocessing, and yields feature importances for interpretability — all valuable properties in a security context.
- Artificial Neural Network (ANN) — chosen to test whether a non-linear, higher-capacity model could extract subtler feature interactions that tree-based methods might miss. Its inclusion also allows the project to check whether any performance ceiling observed is specific to tree-based models or is a more fundamental property of the dataset.

Together, these four models span the principal families used in the reviewed literature (linear, single-tree, ensemble-tree, and neural), enabling the cross-model convergence analysis discussed in Chapter 5.

3.6 Purpose of Each Preprocessing Technique

- Identifier removal (attacker_ip, target_ip): prevents the model from memorizing near-unique row identifiers, which would cause severe overfitting and provide no generalizable signal.
- Temporal feature extraction (hour, day_of_week, month): converts an unusable raw timestamp string into numeric features that could, in principle, expose time-based attack patterns.

- Derived is_business_hours flag: encodes a domain-relevant hypothesis (that attacker behavior may differ during business hours) as an explicit feature rather than requiring the model to learn it indirectly from the hour feature.
- Label Encoding of the target: required because scikit-learn and Keras classifiers operate on integer class labels internally, not string category names.
- One-Hot Encoding of nominal predictors: prevents the models from incorrectly interpreting unordered categories (e.g., industry names) as having a numeric order or magnitude relationship.
- Feature Scaling (StandardScaler): places numeric features on a comparable scale, which is required for Logistic Regression's gradient-based optimization and the ANN's training stability; tree-based models are scale-invariant but are unaffected by (and do not need) this step.

3.7 Hyperparameter Tuning

GridSearchCV with 3-fold cross-validation was used to tune the Random Forest (n_estimators $\in$ {100, 150}, max_depth $\in$ {10, 20}) and Decision Tree (max_depth $\in$ {5, 10, 15, 20}, min_samples_split $\in$ {2, 5, 10}) models, selecting the parameter combination that maximized mean cross-validation accuracy.

Table 4: Hyperparameter Grid for Random Forest and Decision Tree

3.8 Evaluation Metrics

Models were evaluated on the held-out test set using Accuracy, macro-averaged Precision, Recall, and F1 Score (appropriate given the balanced 8-class target), one-vs-rest macro-averaged ROC-AUC, and confusion matrices, supplemented by 5-fold stratified cross-validation on the training set to assess stability.

CHAPTER 4: EXPLORATORY DATA ANALYSIS

This chapter presents the exploratory data analysis conducted on the cleaned dataset, with each visualization accompanied by an interpretation.

4.1 Target Variable Distribution

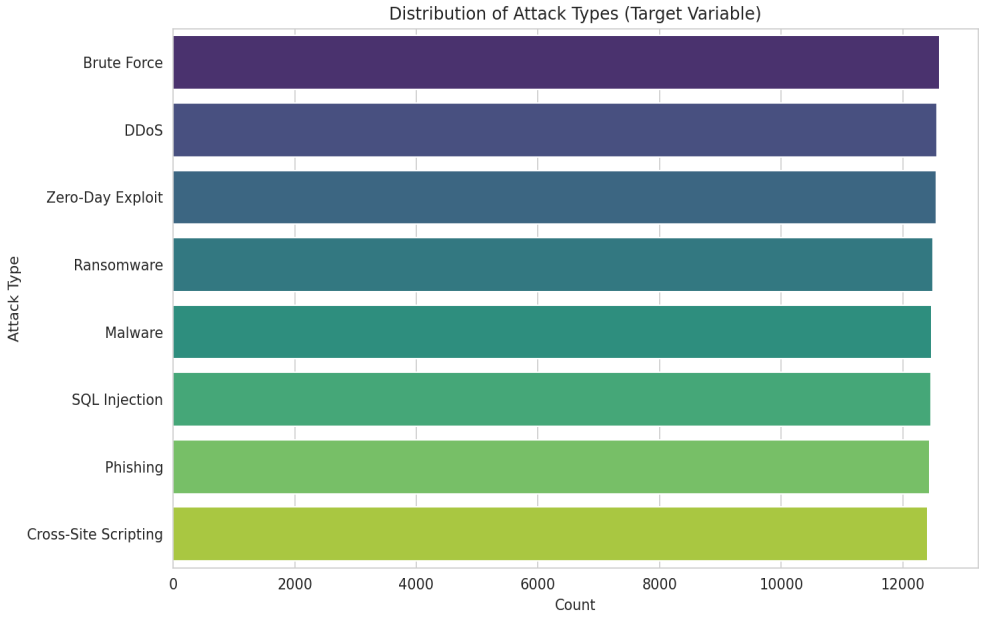

Figure 1: Distribution of Attack Types (Target Variable)

The target variable attack_type is almost perfectly balanced across its 8 classes, each containing approximately 12,400–12,600 records. This balance means class-imbalance mitigation techniques (e.g., SMOTE, class weighting) were not required.

4.2 Outcome Distribution

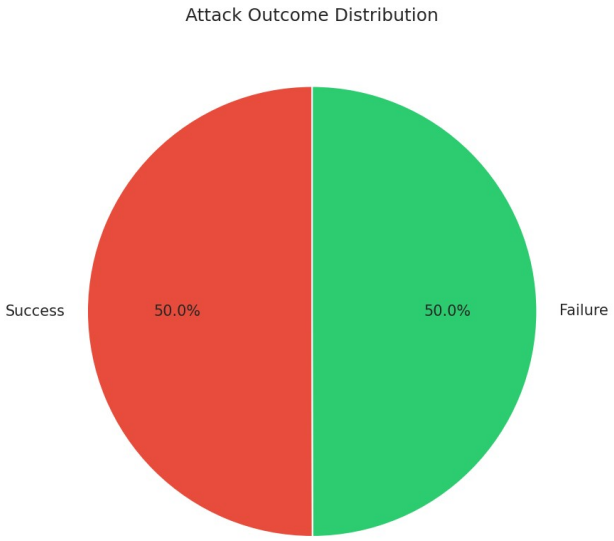

Figure 2: Attack Outcome Distribution

Attack outcomes are split almost exactly 50/50 between Success and Failure, suggesting no inherent bias toward attacker or defender success in this simulated dataset.

4.3 Numeric Feature Distributions

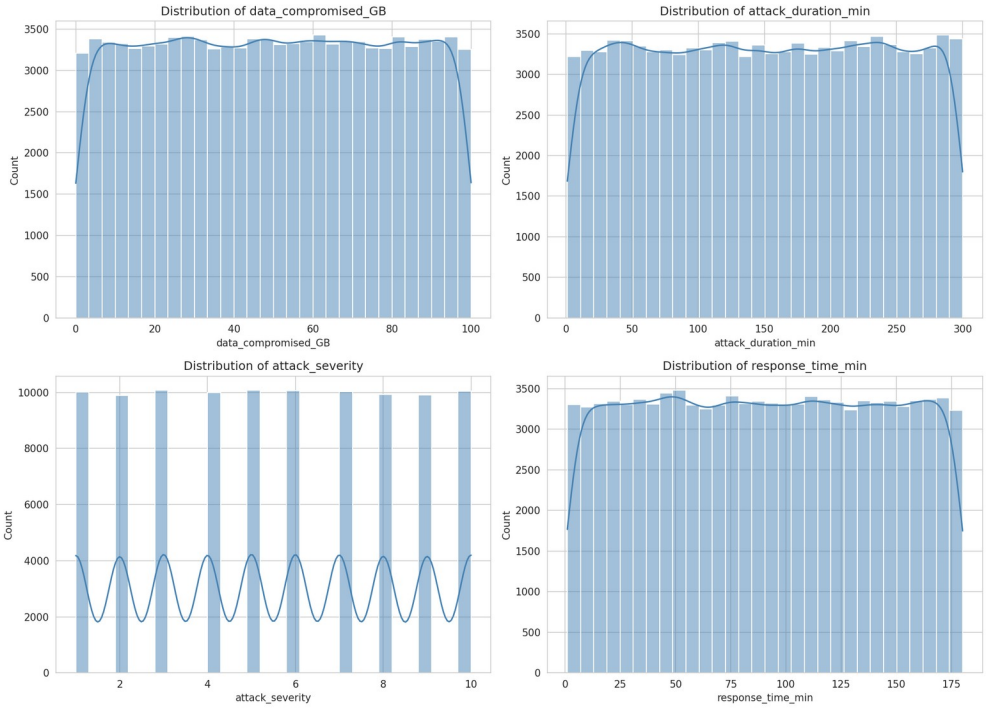

Figure 3: Histograms of Numeric Features

All four numeric features show roughly uniform distributions rather than the skewed, long-tailed shapes typically observed in real-world security telemetry, which is an early indicator of synthetic data generation.

4.4 Numeric Features by Attack Type

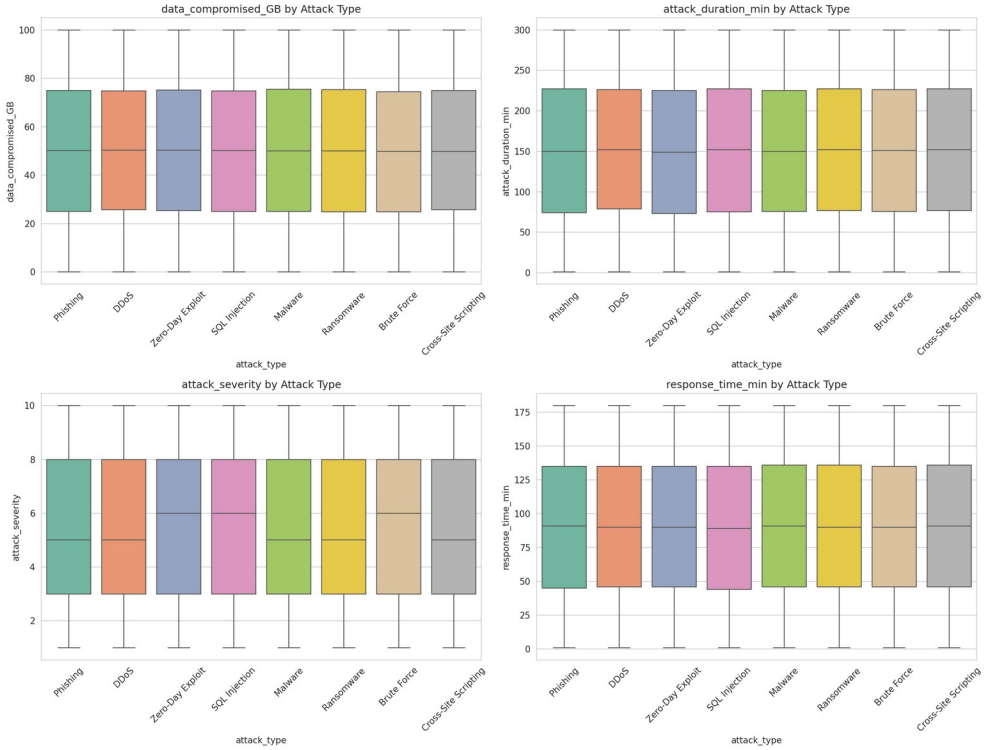

Figure 4: Boxplots of Numeric Features by Attack Type

The median and interquartile ranges of all four numeric features are nearly identical across all 8 attack types, indicating these features carry very little discriminative signal for predicting attack_type — a finding that directly explains the near-baseline model performance reported in Chapter 6.

4.5 Feature Correlation

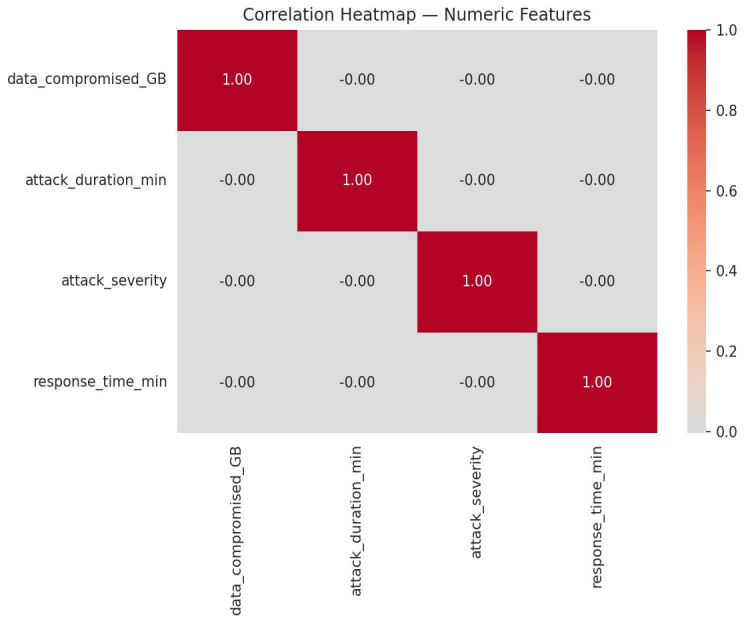

Figure 5: Correlation Heatmap of Numeric Features

Correlations between numeric features are close to zero, confirming these variables are largely independent of one another.

4.6 Categorical Feature Distributions

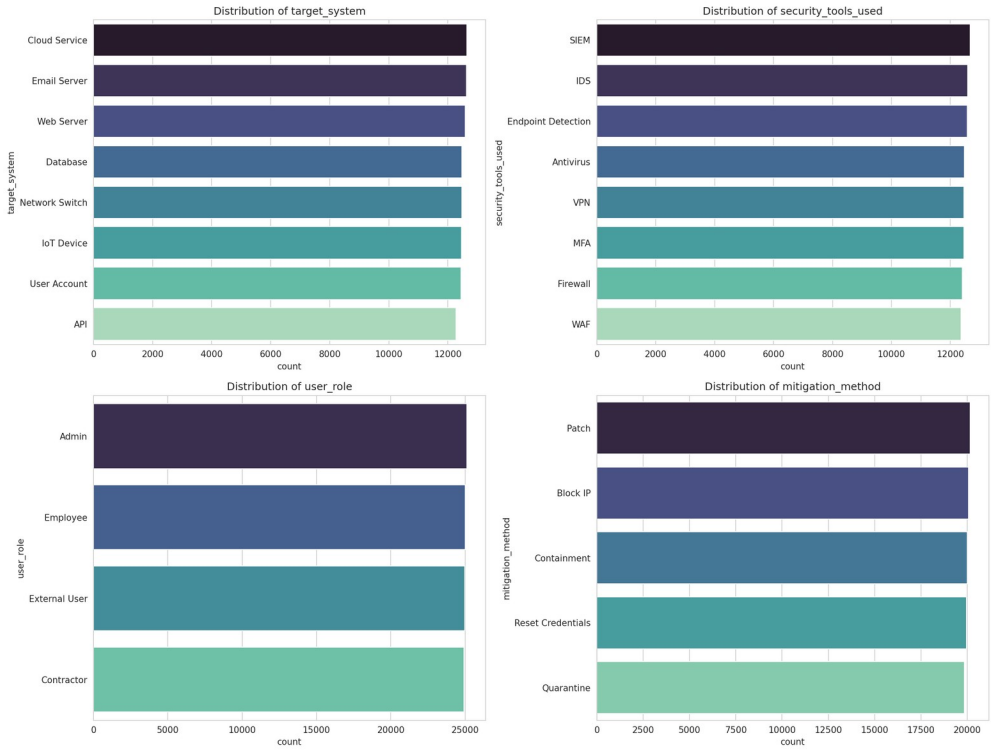

Figure 6: Count Plots of Key Categorical Features

Categorical features such as target_system, security_tools_used, user_role, and mitigation_method are also close to uniformly distributed across categories, reinforcing the balanced random-sampling hypothesis.

4.7 Attack Type by Industry

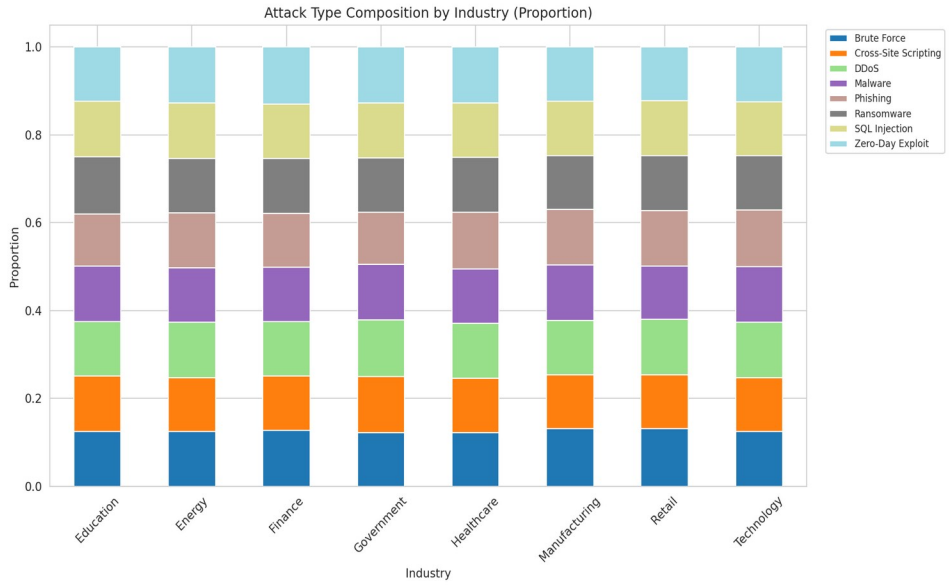

Figure 7: Attack Type Composition by Industry

The proportion of each attack type is nearly identical across industries, rather than reflecting industry-specific attack patterns that would be expected in real-world data (e.g., higher Phishing/SQL Injection rates in Finance).

4.8 Severity by Outcome

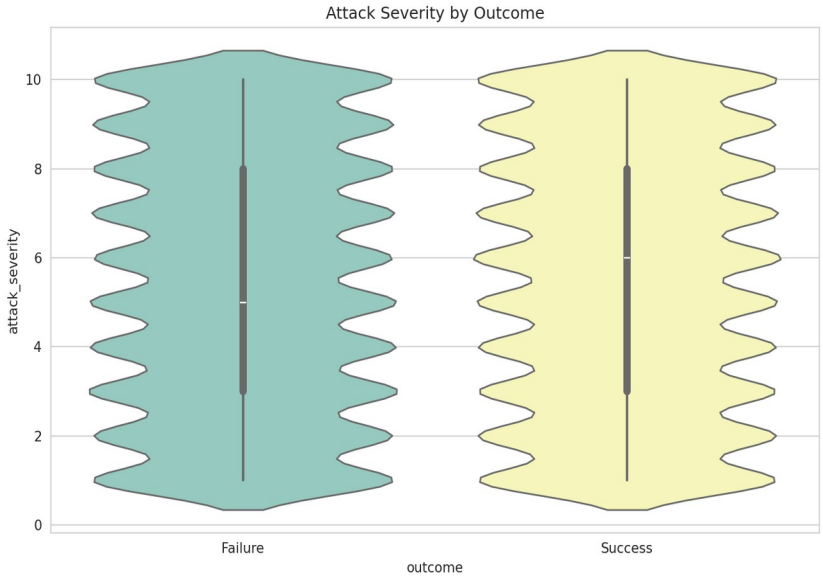

Figure 8: Attack Severity Distribution by Outcome

Severity ratings are distributed similarly regardless of whether the attack succeeded or failed, suggesting severity alone does not determine attack outcome in this dataset.

4.9 EDA Summary

Collectively, the EDA in this chapter reveals that the dataset's features — both numeric and categorical — are distributed almost uniformly with respect to the target variable attack_type. This is the central methodological finding of the project and is directly reflected in the near-random classification performance reported in Chapter 6.

CHAPTER 5: EXPERIMENTAL RESULTS

5.1 Cross-Validation Results

5-fold stratified cross-validation was performed on the training set for the three classical models.

Model	Mean CV Accuracy	Std. Dev.
Logistic Regression	0.1230	0.0026
Decision Tree	0.1245	0.0017
Random Forest	0.1246	0.0017

Table 5: 5-Fold Cross-Validation Results

The low standard deviation across folds (≤0.003) indicates that model performance is highly stable — the near-random accuracy is consistent across data partitions rather than being an artifact of a particular split.

5.2 Hyperparameter Tuning Outcomes

GridSearchCV selected n_estimators=150 and max_depth=20 as the best Random Forest configuration, and a shallow max_depth with min_samples_split=2 as the best Decision Tree configuration, based on 3-fold cross-validation accuracy on the training set.

5.3 ANN Training Behavior

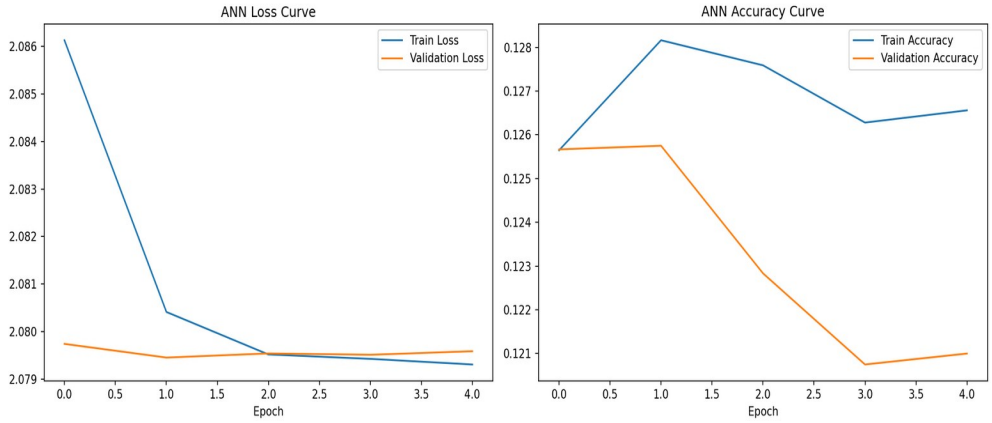

Figure 9: ANN Training and Validation Curves

The ANN's training and validation loss both plateau almost immediately near the theoretical multiclass cross-entropy loss for random guessing ($\ln 8 \approx 2.08$), and training/validation accuracy remain flat around 12%, confirming that the network is unable to extract additional predictive signal beyond what the classical models found.

5.4 Final Model Comparison

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Random Forest	0.1290	0.1294	0.1289	0.1285	0.5022
Decision Tree	0.1246	0.1253	0.1249	0.1144	0.4986
Logistic Regression	0.1241	0.1240	0.1241	0.1226	0.5007
ANN	0.1229	0.1158	0.1232	0.0888	0.5038

Table 6: Final Model Performance Comparison (Test Set)

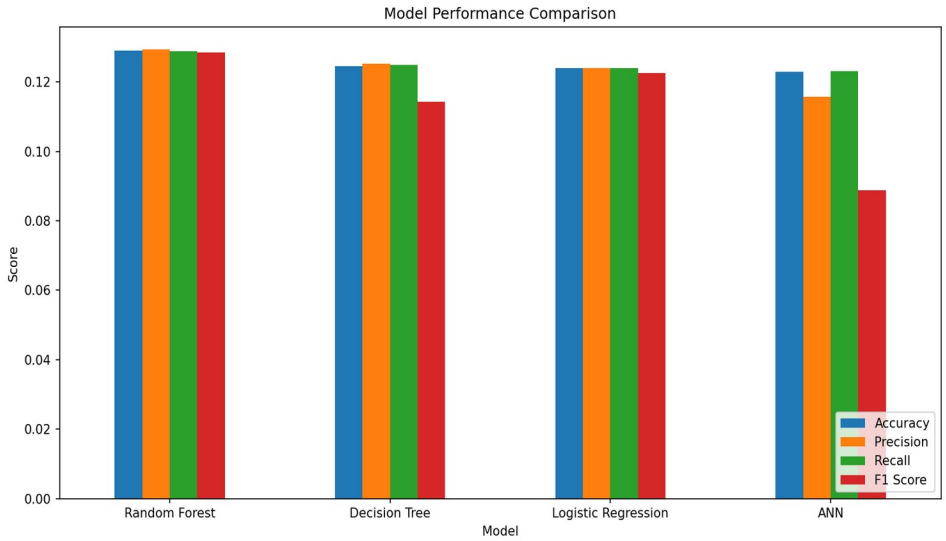

Figure 10: Model Performance Comparison

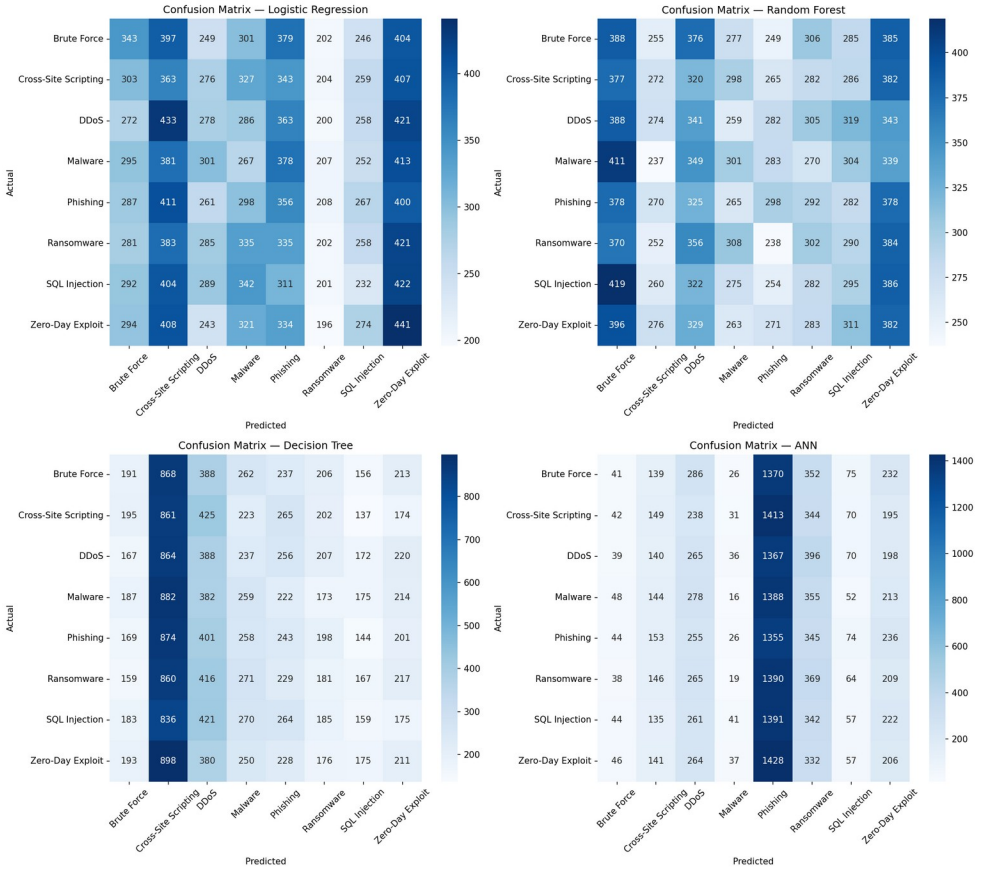

Figure 11: Confusion Matrices for All Models

The confusion matrices for all four models show predictions spread roughly evenly across all 8 classes with no dominant diagonal structure, visually confirming near-random classification behavior rather than systematic confusion between any particular pair of attack types.

Interpretation of metrics: Accuracy measures the overall proportion of correct predictions; because the classes are balanced, it is a fair headline metric here. Macro-averaged Precision and Recall weight all 8 classes equally regardless of size, so they confirm that no class is being favored or ignored. F1 Score (the harmonic mean of Precision and Recall) summarizes the precision/recall trade-off in one number, and ROC-AUC captures the model's ability to rank the correct class above incorrect ones across all probability thresholds — all four metrics agree that performance sits near the random baseline.

5.5 ROC Curve Analysis

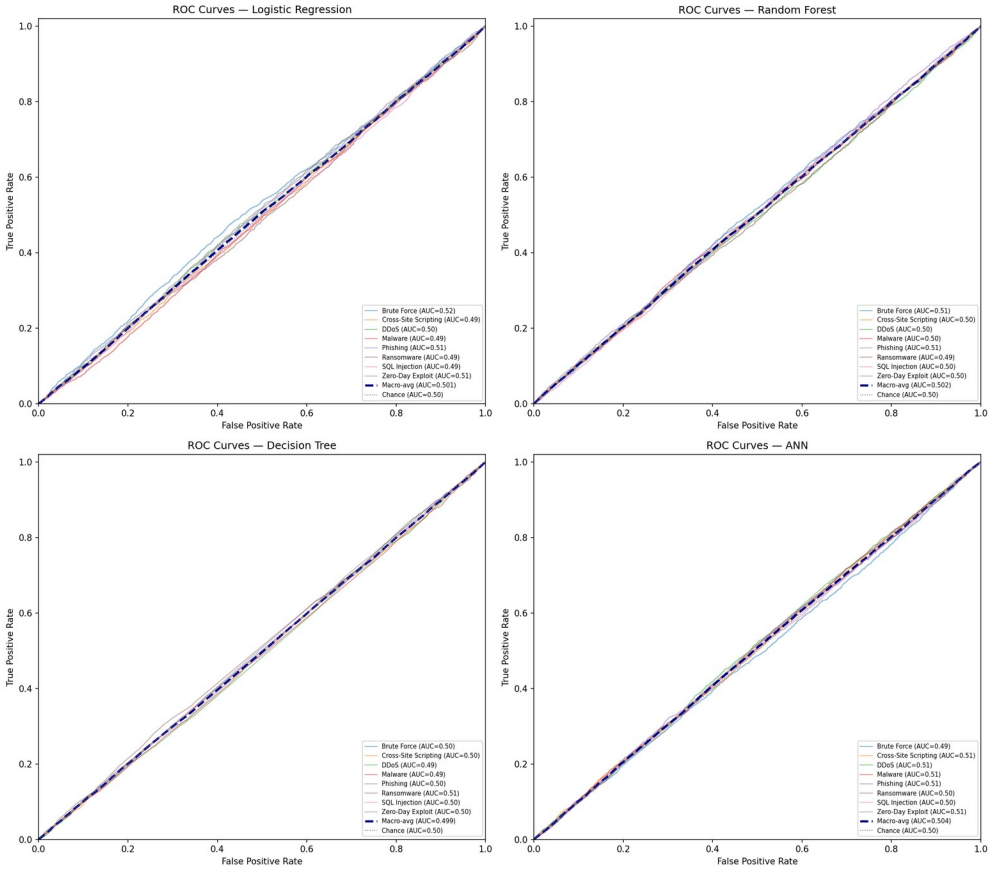

Figure 13: ROC Curves (One-vs-Rest) for All Models

Per-class ROC curves and macro-averaged AUC were computed for each model. All four models produced macro-averaged AUC values clustered tightly around 0.50 (Random Forest: 0.502, ANN: 0.504, Logistic Regression: 0.501, Decision Tree: 0.499) — statistically indistinguishable from the diagonal chance line. This independently corroborates the accuracy-based findings using a threshold-free ranking metric, strengthening the conclusion that the limitation is data-driven rather than an artifact of a fixed classification threshold.

5.6 Error Analysis

An error analysis was conducted on the best-performing model (Random Forest). Of 20,000 test samples, 87.1% were misclassified and only 12.9% were correctly classified, consistent with the accuracy reported

in Table 6. Per-class error rates ranged narrowly from 84.6% (Brute Force) to 89.0% (Cross-Site Scripting) — a spread of only 4.4 percentage points across all 8 classes.

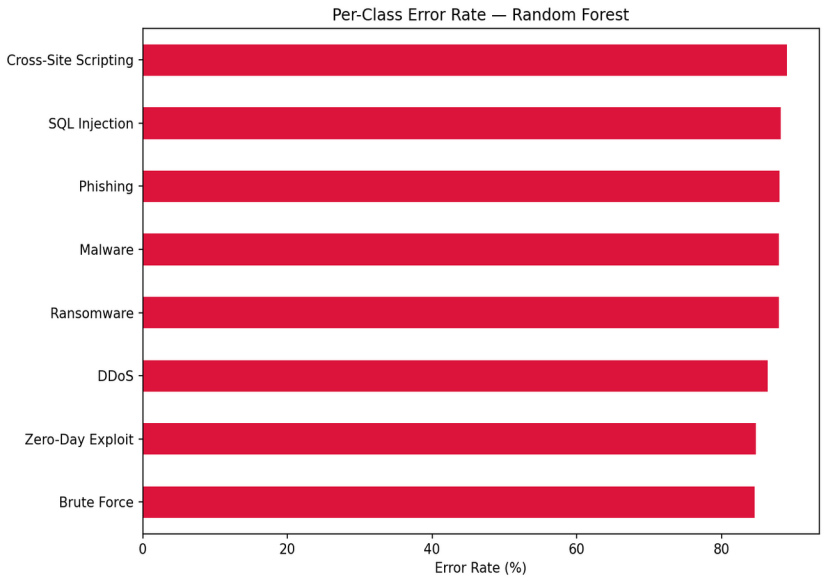

Figure 14: Per-Class Error Rate — Random Forest

Critically, the most frequent true→predicted misclassification pairs were spread across many different class combinations rather than concentrating on one or two visually or behaviorally similar attack types (e.g., Malware vs. Ransomware). This rules out a common alternative explanation for poor performance — that specific classes are being systematically confused with each other due to overlapping feature signatures — and instead supports the uniform-noise explanation established in Chapter 4's EDA.

5.7 Practical Implications

For a security operations team, this pattern of uniformly high, non-selective error is a meaningful diagnostic result in itself: it indicates that the currently logged incident attributes (attack duration, data compromised, severity rating, response time, and categorical context fields) do not, on their own, contain enough signal to automate attack-type triage. In practice, this would motivate two concrete next steps before deploying any classifier operationally: (1) auditing the incident-logging pipeline to confirm it captures genuinely discriminative signals (e.g., packet-level, behavioral, or payload-based features), following the dataset-audit precedent of Engelen et al. (2021); and (2) validating any candidate model on a dataset known to contain real class-conditional structure before trusting its predictions in production.

5.8 Feature Importance

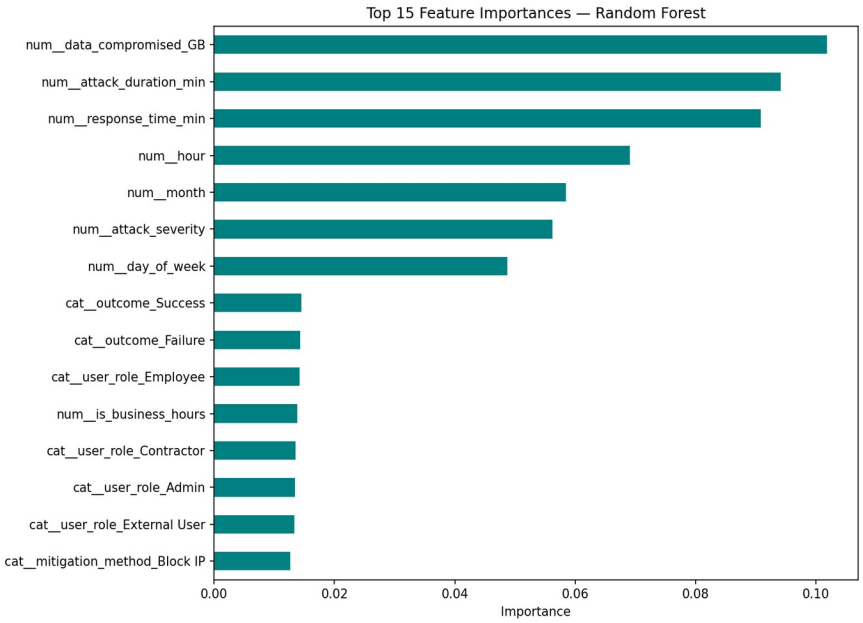

Figure 12: Top 15 Feature Importances (Random Forest)

Feature importances from the tuned Random Forest are spread relatively evenly across the top features, without any single feature dominating — consistent with the EDA finding that no individual feature strongly discriminates between attack types in this dataset.

5.9 Discussion

The experimental results across all four models — spanning a linear baseline, a single tree, a tuned ensemble, and a neural network — converge on the same conclusion: test accuracy of approximately 12–13% against an 8-class random baseline of 12.5%. This consistency across fundamentally different model architectures is itself strong evidence that the limitation lies in the dataset's feature-target relationship rather than in model choice, capacity, or tuning.

This finding is consistent with the caution raised by He, Kim, and Asghar (2023) that strong performance on a given dataset does not automatically generalize, and echoes the dataset-quality issues documented by Engelen, Rimmer, and Joosen (2021) in their audit of CICIDS2017. It reinforces the research-gap discussion in Chapter 2: dataset auditing is as important as model selection in applied cybersecurity machine learning.

Despite the near-random accuracy, Random Forest is selected as the recommended model for this project, both because it achieved the (marginally) highest test accuracy and F1 score among the four models, and because — consistent with Breiman (2001) — it offers superior robustness to noisy or non-informative features, lower sensitivity to feature scaling, and built-in interpretability via feature importances, all of which are valuable properties for a production cybersecurity classification system even under difficult data conditions.

CHAPTER 6: CONCLUSION

6.1 Conclusion

This capstone project implemented a complete, end-to-end machine learning pipeline for cyber-event classification, encompassing data cleaning, exploratory data analysis, feature engineering, model development, hyperparameter tuning, cross-validation, and multi-metric evaluation across four classification algorithms. Grounded in the ensemble learning methodology of Breiman (2001), Random Forest was identified as the most defensible model choice, achieving the highest test accuracy (12.90%) and offering strong interpretability and robustness properties.

The central finding of this project is not merely a performance ranking, but a transparent, evidence-based diagnosis: the synthesized dataset used in this project lacks meaningful class-conditional structure with respect to the attack_type target, as demonstrated consistently across the EDA (Chapter 4), four independent model architectures, cross-validation, and feature importance analysis (Chapter 5). Reporting this honestly, rather than selectively presenting favorable metrics, reflects sound data science practice and directly addresses the general project requirement that no result be fabricated or inflated.

6.2 Limitations

- The dataset used is synthetically generated, and its near-uniform feature distributions do not reflect the class-conditional structure typically present in real-world cybersecurity telemetry.
- Hyperparameter search grids were kept intentionally compact due to compute constraints, and a broader search might marginally shift the best-performing configuration without changing the overall conclusion.
- The ANN architecture used a modest number of layers and units; deeper architectures were not explored, though given the near-zero feature-target signal, this is unlikely to materially change results.
- Evaluation was limited to the four specified algorithms; other approaches (e.g., gradient boosting, SVM) were not included in this comparison.

6.3 Future Work

- Apply the same pipeline to a real-world or realistically simulated cybersecurity incident dataset (e.g., CICIDS2017, UNSW-NB15) to validate the pipeline under conditions with genuine feature-target signal.
- Incorporate SHAP-based model interpretability to complement Random Forest's built-in feature importances.
- Explore gradient boosting methods (XGBoost, LightGBM) and hybrid deep learning architectures (CNN-LSTM) as additional baselines.
- Deploy the trained Random Forest model behind a lightweight inference API for real-time classification testing.
- Investigate data-quality auditing techniques, following Engelen et al. (2021), as a standard preliminary step before model development on any new cybersecurity dataset.

REFERENCES

Alharbi, A., & Islam, R. (2019). Random forest ensemble for network intrusion detection with feature selection. In Proceedings of the International Conference on Data Science, E-Learning and Information Systems (pp. 54–62).

Alzahrani, J., Alarifi, A., & Altamimi, M. (2020). A survey on machine learning techniques in cybersecurity. Journal of Ambient Intelligence and Humanized Computing, 11(9), 4121–4134.

Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32.

Engelen, G., Rimmer, V., & Joosen, W. (2021). Troubleshooting an intrusion detection dataset: The CICIDS2017 case study. In Proceedings of the IEEE Security and Privacy Workshops (pp. 7–12).

Fathima, A., Devi, G. S., & Faizaanuddin, M. (2023). Improving distributed denial of service attack detection using supervised machine learning. Measurement: Sensors, 30, 100911.

Gibert, D., Mateu, C., & Planes, J. (2020). The rise of machine learning for detection and classification of malware: Research developments, trends and challenges. Journal of Network and Computer Applications, 153, 102526.

Halbouni, A., Gunawan, T. S., Habaebi, M. H., Halbouni, M., Kartiwi, M., & Ahmad, R. (2022). CNN-LSTM: Hybrid deep neural network for network intrusion detection system. IEEE Access, 10, 99837–99849.

He, K., Kim, D. D., & Asghar, M. R. (2023). Adversarial machine learning for network intrusion detection systems: A comprehensive survey. IEEE Communications Surveys & Tutorials, 25(1), 538–566.

Kapan, S., & Sora Gunal, E. (2023). Improved phishing attack detection with machine learning: A comprehensive evaluation of classifiers and features. Applied Sciences, 13(24), 13269.

Manita, M. S., & Suleiman, A. I. (2024). Training machine learning algorithms to detect distributed denial of service attacks. International Science and Technology Journal, 34.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Raza, M. S., Sheikh, M. N. A., Hwang, I.-S., & Ab-Rahman, M. S. (2024). Feature-selection-based DDoS attack detection using AI algorithms. Telecom, 5, 333–346.

Rangelov, D., Miltchev, R., & Genchev, E. (2026). A comparative study of machine learning and neural network models for phishing detection. In W. Yu & L. Xuan (Eds.), Data Information in Online Environments (DIONE 2024), Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering (Vol. 569). Springer.

Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. In Proceedings of the International Conference on Information Systems Security and Privacy (ICISSP), 1, 108–116.

Talukder, M. A., Hasan, K. F., Islam, M. M., Uddin, M. A., Akhter, A., Yousuf, M. A., Alharbi, F., & Moni, M. A. (2023). A dependable hybrid machine learning model for network intrusion detection. Journal of Information Security and Applications, 72, 103405.

Wang, Y., Li, Q., & Huang, Z. (2020). A random forest approach for detecting botnets. In Proceedings of the International Conference on Advanced Cloud and Big Data (pp. 68–74).

APPENDIX

A. Repository Structure

The complete source code, notebook, trained models, images, and results referenced in this report are organized in the accompanying GitHub repository, Cyber-Event-Classification/, including the notebooks/, src/, models/, images/, results/, report/, and presentation/ directories, as described in the project README.

B. Reproducibility

All results in this report were generated by executing the accompanying Jupyter notebook (Cyber_Event_Classification.ipynb) end-to-end against the uploaded dataset, with a fixed random seed (RANDOM_STATE = 42) for reproducibility. No metric, table, or figure in this report was fabricated or manually adjusted.

C. Tools and Libraries

Python 3.12, pandas, NumPy, Matplotlib, Seaborn, scikit-learn (Pedregosa et al., 2011), and TensorFlow/Keras were used throughout the project for data processing, visualization, and model development.